PROJET DE SCIENCE DE L'INGÉNIEUR

Hélice 31

LIVRABLE TECHNIQUE

De la découverte des composants jusqu'à la maquette physique

Équipe Projet	Informations
Tristan BERNARD **Fatihhan COG** **Nils COUDRY** **Loan BOUYAHI** **Louis QUIBEUF** **Victor BON** **Arthur KEMPNICH**	Domaine : Science de l'ingénieur (SI) Durée : 5 séances Simulation : Matériel Principal : Arduino Uno

Ce document est un rapport d'avancement détaillé de notre projet.

Sommaire

Section	Titre	Page
0	Page de Garde	1
Sommaire	Sommaire	2
1	Contexte	3
2	La théorie — Comprendre avant de construire	4
2.1	Le Cerveau et L'Échantillonnage : La carte Arduino	4
2.2	Les Sens : Le Gyroscope / Accéléromètre	4
2.3	Les Actionneurs : Moteur Brushless et ESC	4
2.4	L'Interface Humain-Machine (IHM) : Potentiomètres et Bouton	5
2.5	La Signalisation : LED et Résistance	5
3	Journal de Bord — Avancement séance par séance	6
4	Architecture Technique du Montage	7
4.1	Composants	7
4.2	Schéma Électronique	8
4.2.1	Version avec TM1637	8
4.2.2	Version avec LED	9
4.3	Montage Réel	9-10
5	Explication du code	11
5.1	Découpage Modulaire du Code	11
5.2	L'Échantillonnage Temps Réel	11
5.3	Modélisation du Correcteur PID Numérique	11-12
5.4	Traitement du Signal	12
5.5	Interface Homme-Machine et Sécurité	12
5.6	Modélisation Numérique et Simulation	13
5.7	Obtention des données avec notre montage	13
6	MatLab	14
6.1	Diagnostic et identification du Système (Boucle Ouverte)	14
6.2	Synthèse du Correcteur (La Compensation de Pôles)	14
6.3	Validation et analyse des résultats	14-15
7	Conclusion	16
8	Note de l'Équipe	17

1. Contexte et Cahier des Charges

Ce projet s'inscrit dans le cadre de la mise en pratique et de la valorisation des compétences de la filière E2. L'objectif principal est de concevoir un démonstrateur technique attractif et transportable, destiné à être exposé lors de salons et de Journées Portes Ouvertes (JPO). Prenant la forme d'une maquette d'hélicoptère d'environ 40 par 40 centimètres (réalisée en impression 3D), ce système a pour but d'illustrer concrètement les principes d'asservissement sur deux axes.

D'un point de vue technique, le système s'affranchit d'un pilotage direct depuis Simulink au profit d'une commande numérique embarquée autonome basée sur une carte Arduino et un gyroscope MPU 6050. Le pilotage s'effectuera via un boîtier de commande radiofréquence. L'un des aspects les plus didactiques de ce démonstrateur est sa capacité à être réglé "en direct" : l'utilisateur pourra ajuster les coefficients du correcteur PID (Kp, Ki, Kd) à l'aide de potentiomètres dédiés afin d'observer immédiatement l'impact sur la stabilité, avec l'aide de LED signalant notamment la saturation de la commande. La boucle d'asservissement reposera sur un échantillonnage rigoureux, utilisant les *timers* (interruptions) pour cadencer la mesure des angles et l'envoi des consignes aux moteurs *brushless*.

Le développement de cette maquette implique également de relever des défis mécaniques et physiques importants. Une double modélisation est nécessaire : la commande du tangage par le rotor principal génère un couple qui fait tourner la structure. Cette rotation parasite doit être compensée par un rotor de queue dont les variations de vitesse permettront de diriger l'appareil. La réussite du projet reposera sur la minimisation des frottements aux jonctions et sur l'optimisation du poids de l'ensemble.

Enfin, la réalisation de ce projet suivra une méthodologie stricte, débutant par un prototypage sur *breadboard*. Pour des raisons évidentes de sécurité, aucun test impliquant les hélices ne sera toléré sans la supervision du professeur (avec une attention particulière portée à l'initialisation des moteurs *brushless*). Le développement autorise l'usage de la documentation en ligne et de l'intelligence artificielle générative, à condition que cette dernière soit utilisée avec discernement (pour éviter les erreurs de génération) et que son usage soit explicitement sourcé. Si le besoin s'en fait sentir, une évolution prévoyant la bascule entre un correcteur analogique et numérique pourra être étudiée en collaboration (tâche attribuable à M. Trenchant).

2. La Théorie — Comprendre Avant de Construire

Pour que le démonstrateur "Hélice-31" puisse s'auto-stabiliser selon ses deux axes (tangage et lacet), il est nécessaire de créer une boucle d'asservissement complète. Cette boucle repose sur des composants matériels précis qui interagissent en temps réel. Voici le rôle théorique et physique de chacun d'entre eux.

2.1 Le Cerveau et l'Échantillonnage : La Carte Arduino

L'Arduino est le microcontrôleur au cœur du système. Son rôle théorique n'est pas seulement de lire des valeurs, mais d'exécuter la **commande numérique**.

Dans un système asservi informatiquement, le temps n'est pas continu mais discret. L'Arduino utilise ses **timers internes et des interruptions** pour définir une période d'échantillonnage stricte (par exemple, toutes les 10 ou 20 ms). À chaque "tic" d'horloge, il exécute la même boucle :

1. Mesure de l'erreur (via le gyroscope).
2. Calcul du correcteur PID.
3. Envoi de la nouvelle consigne aux moteurs.
   Cette régularité est indispensable pour que les calculs mathématiques d'intégration (pour le Ki) et de dérivation (pour le Kd) soient justes.

2.2 Les Sens : Le Gyroscope / Accéléromètre

Le MPU6050 est une centrale à inertie (IMU) qui permet de mesurer l'angle réel de la maquette.

- **Le Gyroscope :** Mesure la vitesse de rotation angulaire (en degrés par seconde). Il est ultra-réactif mais accumule des erreurs dans le temps (dérive).
- **L'Accéléromètre :** Mesure le vecteur de la gravité terrestre pour donner un angle absolu par rapport au sol. Il est très sensible aux vibrations des moteurs.
- **Théorie d'utilisation :** Le code de l'Arduino devra utiliser un filtre mathématique (souvent un "Filtre Complémentaire") pour fusionner ces deux données et obtenir un angle stable et précis, qui servira de retour pour la boucle d'asservissement.

2.3 Les Actionneurs : Moteur Brushless et ESC

Contrairement à un petit moteur à courant continu basique, un moteur "Brushless" (sans balais) est un moteur triphasé. Il ne peut pas fonctionner en le branchant directement sur une batterie.

- **L'ESC (Electronic Speed Controller) :** C'est le variateur électronique. Il transforme le courant continu de l'alimentation en impulsions triphasées pour faire tourner le moteur.
- **Contrôle et Sécurité :** L'ESC se pilote via un signal PWM (généré par l'Arduino). **Point critique de spécification :** Par sécurité absolue (surtout lors de l'utilisation d'hélices), un ESC possède une sécurité au démarrage. Il exige de recevoir un signal correspondant à "0% de puissance" pendant quelques secondes à l'allumage pour s'armer, sinon il bloque le moteur.

2.4 L'Interface Humain-Machine (IHM) : Potentiomètres et Bouton

Le boîtier de commande permet d'interagir avec le système numérique en temps réel, sans avoir à recompiler le code.

- **Le Potentiomètre :** Sur le plan théorique, c'est un pont diviseur de tension. L'Arduino lit une tension analogique comprise entre 0V et 5V, qu'il convertit en une valeur numérique (de 0 à 1023).
  Vout = Vin(R2/(R1+R2)*
  Dans notre projet, ils auront deux rôles : définir la consigne de vol (le point d'équilibre désiré), et surtout, régler en direct les coefficients Kp, Ki et Kd de l'asservissement.
- **Le Bouton poussoir :** Il servira d'entrée numérique logique (0 ou 1) pour déclencher des actions spécifiques (réinitialisation des intégrateurs du PID, bascule entre modes analogique/numérique, ou arrêt d'urgence).

2.5 La Signalisation : LED et Résistance

Les LEDs permettront un diagnostic visuel rapide de l'état du système (ex: LED allumée si la commande sature, signifiant que le moteur est à 100% de ses capacités mais n'arrive pas à atteindre la consigne).

- **La Résistance de protection :** Une LED est un composant non linéaire qui ne limite pas le courant qui la traverse. Sans résistance, elle grillerait instantanément. La valeur de la résistance est calculée grâce à la loi d'Ohm pour limiter ce courant :
  *U = R*I*
  (Où U est la tension aux bornes de la résistance, et I le courant souhaité pour la LED).

3. Journal de Bord — Avancement Séance par Séance

Séance 1	**Découverte du projet** - Découverte du projet et du cahier des charges. - Attribution des tâches au sein de l'équipe et découverte du matériel. - Premières idées et ébauche du schéma de principe (sur papier).
Séance 2	**Etude du matériel** - Étude technique approfondie du matériel. - Début du schéma électronique. - Initialisation de l'architecture du code Arduino.
Séance 3	**Finalisation schéma et début montage** - Finalisation du schéma électronique et de ses variations. - Début du montage matériel sur breadboard. - Amélioration et développement de la logique du code.
Séance 4	**Finalisation schéma et code** - Prototype matériel terminé. - Tests de validation des signaux et du code sans hélice pour des raisons de sécurité. - Compilation et rédaction de la version finale du livrable.
Séance 5	**Soutenance** - Présentation orale devant jury. - Démonstration du montage fonctionnel.

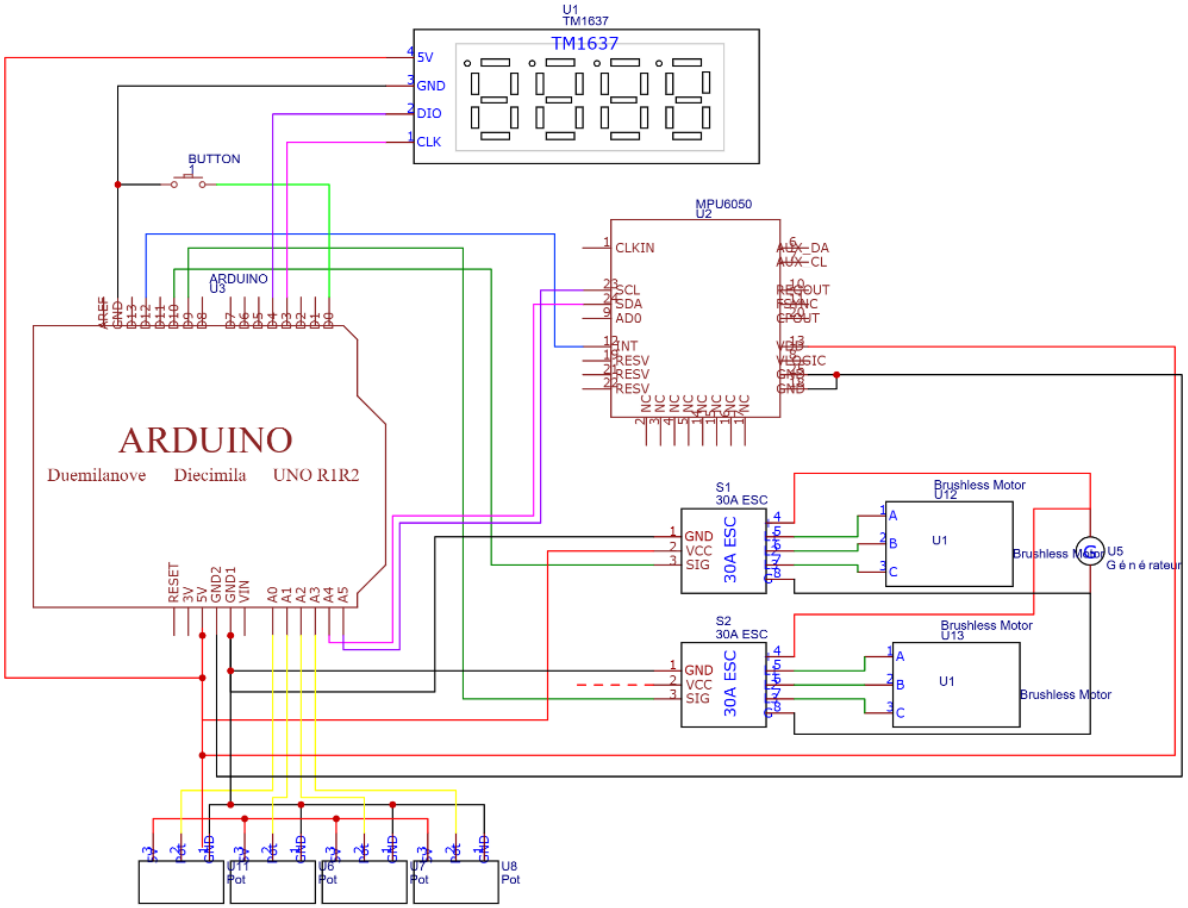
4. Architecture Technique du Montage

4.1 Composant

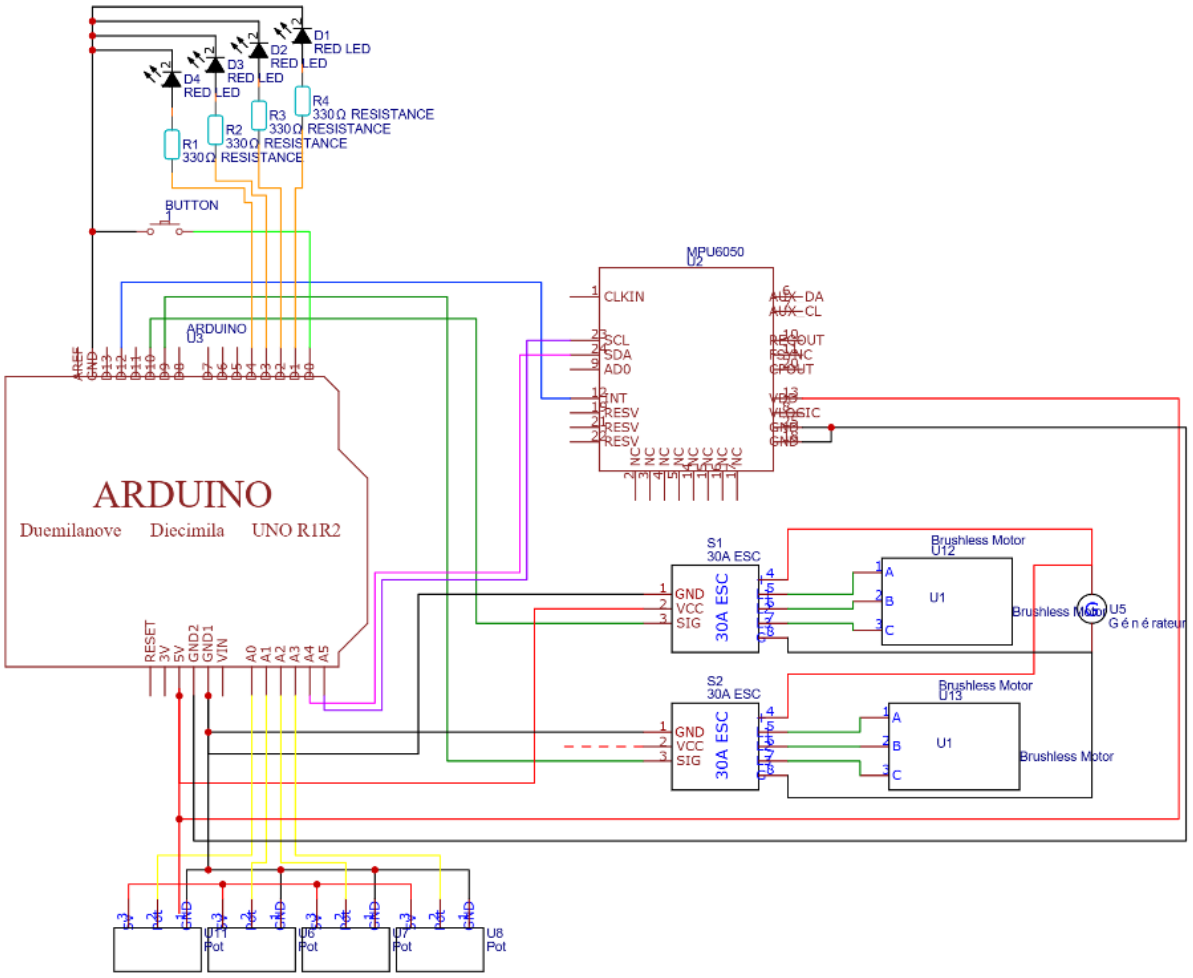
Composant	Rôle
Carte Arduino	Le cerveau du système. Elle exécute le code en temps réel, lit les données des capteurs et des potentiomètres, calcule l'asservissement et envoie les ordres de puissance aux moteurs
Moteur Brushless	L'actionneur mécanique. Il transforme l'énergie électrique en une rotation mécanique à très haute vitesse. Il ne possède pas de frottements internes (pas de balais), ce qui le rend ultra-réactif pour stabiliser la maquette.
ESC	Le traducteur de puissance. L'Arduino ne peut pas alimenter directement un gros moteur. L'ESC reçoit le signal de commande faible (PWM) de l'Arduino et libère l'énergie correspondante du générateur vers le moteur sous forme de courant triphasé.
Potentiomètre	L'interface de commande. Il agit comme un diviseur de tension variable. Il permet à l'opérateur de donner une consigne de vol ou de régler en direct les coefficients mathématiques du système.
LED	L'indicateur de diagnostic. Elle fournit un retour visuel instantané (ex: allumée quand le système est initialisé, clignotante lors d'une saturation de commande ou d'une erreur technique).
Bouton	L'entrée logique (0 ou 1). Dans notre projet, le bouton poussoir ne sert pas uniquement de simple interrupteur, mais de sélecteur de mode (bouton modal). Cette approche logicielle permet de maximiser l'utilisation d'un seul potentiomètre pour contrôler plusieurs paramètres du système.
Gyroscope (MPU-92.65) *(Changement par manque de matériel)*	Le capteur d'attitude. Il mesure en permanence l'accélération terrestre et la vitesse de rotation de la maquette. Il permet au système de connaître son angle exact d'inclinaison physique pour fermer la boucle d'asservissement.
Hélice	Le convertisseur aérodynamique. Elle transforme la rotation mécanique du moteur Brushless en force de poussée. C'est elle qui crée la force physique permettant au bras articulé de vaincre la gravité.
BreadBoard	Le support de prototypage. Elle permet de connecter tous les composants électroniques de manière sécurisée et rapide, sans aucune soudure, ce qui est idéal pour faire des tests ou corriger le schéma.
Cable	Les fils assurent deux missions : transporter l'énergie électrique (VCC, GND) et acheminer les signaux d'information (données des capteurs, signaux PWM) entre la carte et les périphériques.
Générateur	La source d'énergie constante. Utilisé à la place d'une batterie, il garantit une tension électrique parfaitement stable pendant les tests, évitant ainsi que la baisse de tension d'une batterie qui se vide ne fausse le comportement des moteurs et les calculs d'asservissement.

4.2 Schéma Électronique

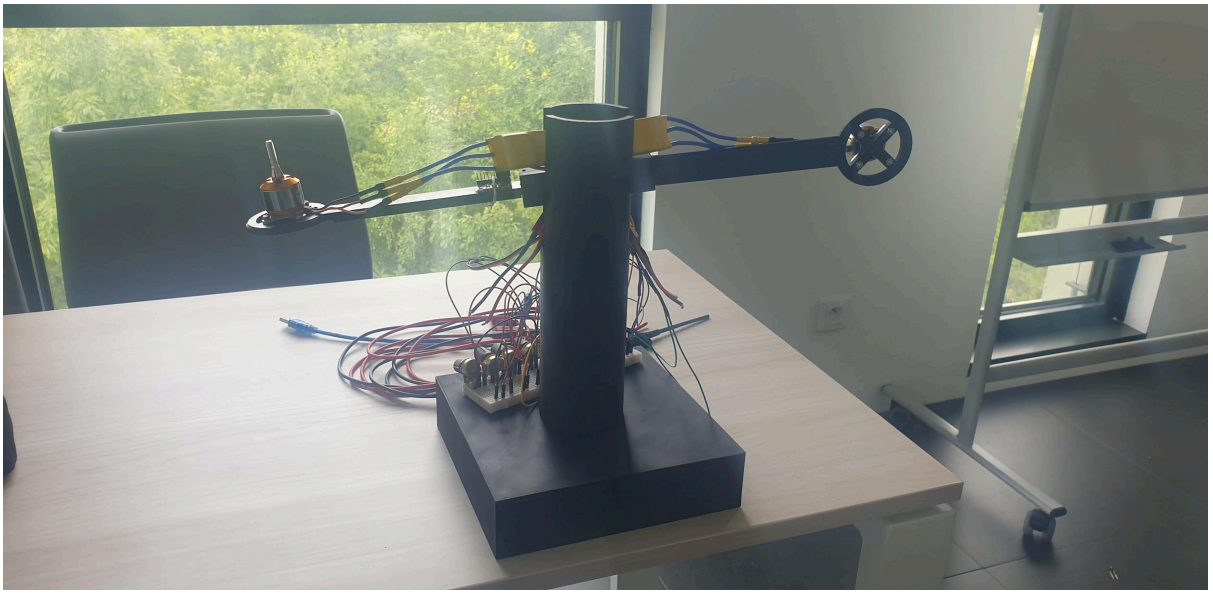
4.2.1 Version avec TM1637 :

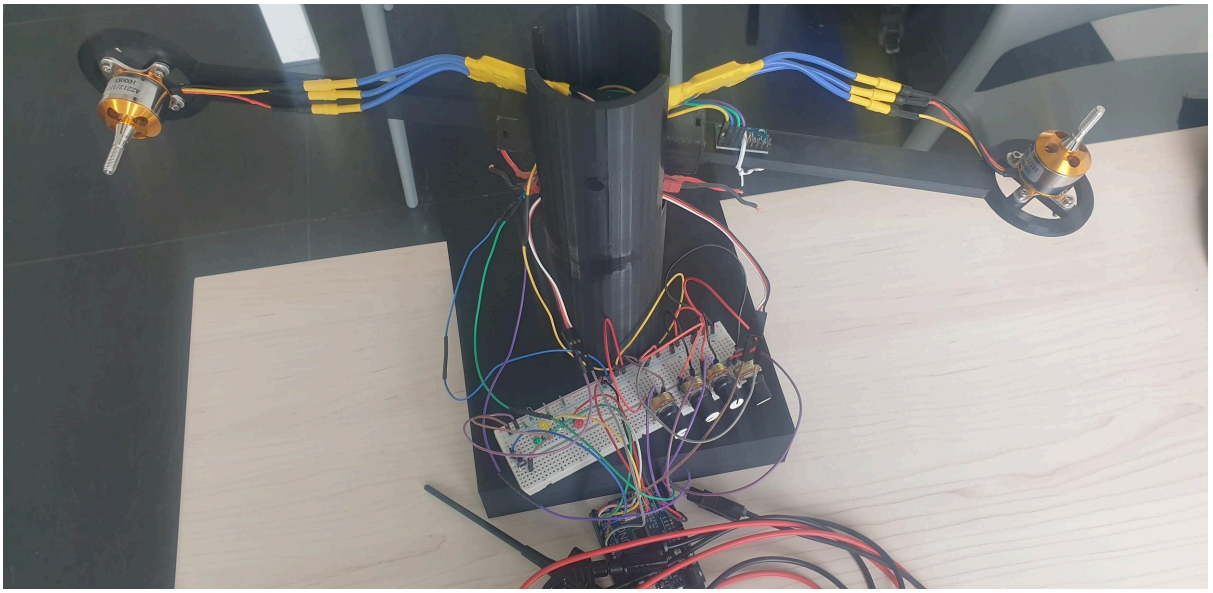


4.2.2 Version avec LED :



4.3 Montage Réel





5. Explication du code

5.1 Découpage Modulaire du Code

Le projet est divisé en plusieurs fichiers distincts pour séparer la configuration, les mathématiques, la lecture des capteurs et le pilotage des actionneurs.

Module	Fichiers	Rôle technique
Programme Principal	Helice31.ino	Contient les boucles setup() et loop(). Il orchestre l'appel de toutes les fonctions selon un ordonnancement précis dicté par les interruptions du Timer.
Configuration	config.h	Regroupe toutes les constantes du système : période d'échantillonnage (Te = 0.1s), affectation des broches (Pins), limites PWM et valeurs maximales des gains.
Capteurs & Filtres	gyro.cpp / .h	Gère la communication I2C avec le MPU-6050 et implémente le filtre complémentaire pour fusionner les données du gyroscope et de l'accéléromètre.
Asservissement	pid.cpp / .h	Cœur mathématique du système. Calcule la commande (Up, Ui, Ud) en fonction de l'erreur, tout en mémorisant l'état précédent.
Actionneurs	moteurs.cpp / .h	Gère la conversion de la commande (0-100) en signal PWM (1000-2000 µs) via la bibliothèque Servo.h et assure la sécurité d'armement des ESC.
Interface Utilisateur	moteurs.cpp / .h	Gèrent la lecture des potentiomètres, la machine à états du bouton (cycle de réglage Kp/Ki/Kd) avec anti-rebond, et le retour visuel par LED.

5.2 L'Échantillonnage Temps Réel

L'une des contraintes majeures de la commande numérique est le respect strict de la période d'échantillonnage (Te = 0.1s). Si la boucle tourne de manière aléatoire, les calculs de dérivation et d'intégration du PID deviennent faux.

Pour éviter cela, nous avons configuré le **Timer 2** de l'Arduino en mode CTC (Clear Timer on Compare Match).

1. Une interruption matérielle (ISR(TIMER2_COMPA_vect)) se déclenche à un intervalle précis.
2. Cette interruption ne fait qu'une seule chose : elle lève un drapeau (flag_echantillon = true).
3. La boucle loop() principale attend que ce drapeau soit levé pour exécuter les calculs critiques (lecture angles, calcul PID, envoi moteurs), garantissant un séquencement temporel rigoureux.

5.3 Modélisation du Correcteur PID Numérique

L'algorithme du correcteur PID a été discrétisé pour fonctionner sur le microcontrôleur. L'erreur est calculée à chaque instant n: erreur = consigne - mesure.

Dans le fichier pid.cpp, les trois actions sont calculées de la manière suivante :

- **Action Proportionnelle (Up) :** Directement liée à l'erreur actuelle.
  *Up = Kp * erreur*
- **Action Intégrale (Ui) :** Utilise la méthode d'intégration trapézoïdale pour limiter les erreurs de calcul numériques. Elle mémorise l'intégrale précédente pour cumuler l'erreur dans le temps.
  $Ui = Ui_{prec} + ((Ki * Te)/2) * (erreur - erreur_{prec})$
- **Action Dérivée Filtrée (Ud) :** La dérivation numérique pure amplifie fortement le bruit des capteurs. Un filtre de constante Nfiltre est appliqué pour adoucir le signal de la dérivée tout en gardant sa réactivité.

La commande finale envoyée aux moteurs correspond à la somme des trois actions : *Un = Up + Ui + Ud*.

5.4 Traitement du Signal

Le signal brut du MPU-6050 n'est pas exploitable tel quel. Les vibrations des moteurs faussent l'accéléromètre, et le gyroscope dérive dans le temps. La fonction `mettreAJourFiltreComp()` implémente un filtre qui s'appuie sur les points forts de chaque capteur :

1. **L'intégration du gyroscope** réagit très vite aux changements d'angle brusques.
2. **L'accéléromètre** donne une référence absolue sur le long terme grâce à la gravité terrestre.
3. Une composante de redondance permet même de basculer sur les capteurs mécaniques (potentiomètres) en cas de défaillance de la communication I2C au démarrage (`gyroDisponible = false`).

5.5 Interface Homme-Machine et Sécurité

Le code a été conçu pour permettre un réglage optimal lors des démonstrations :

- **Multiplexage des commandes :** Afin de limiter le câblage, un seul potentiomètre sert à régler les gains. Le bouton poussoir permet de cycler la sélection (Kp -> Ki -> Kd -> Kp). La fonction majSelectionGain() intègre un système d'anti-rebond temporel (Debounce de 50ms) pour éviter les sauts intempestifs.
- **Diagnostic Visuel :** Des LED informent l'utilisateur du paramètre en cours de réglage. Une LED rouge dédiée est gérée par la fonction estEnSaturation() ; elle s'allume instantanément si l'algorithme demande une puissance moteur en dehors des limites physiques (inférieure à 0 ou supérieure à la limite maximale fixée).
- **Sécurité Matériel (ESC) :** Au démarrage (initMoteurs()), le code impose un blocage absolu du programme de 3 secondes en envoyant un signal PWM minimum (1000 µs). Cette séquence est vitale et exigée par les contrôleurs Brushless pour déverrouiller la rotation.

5.6 Modélisation Numérique et Simulation

En début de projet, l'absence de capteur inertiel matériel nous a conduits à développer un simulateur mathématique en boucle ouverte pour ne pas bloquer notre avancée. Ce code d'attente modélisait la physique des axes de tangage et de lacet directement sur l'Arduino.

Le programme utilisait la méthode d'intégration numérique d'Euler : il traduisait la consigne des potentiomètres en force, puis calculait en temps réel l'accélération, la vitesse, et enfin l'angle de la maquette. Pour être réaliste, le modèle prenait en compte l'inertie, les frottements, la gravité terrestre, le couplage entre les axes et l'effet gyroscopique.

Cependant, l'intégration d'Euler accumule inévitablement des erreurs de calcul, provoquant une dérive de l'angle estimé après une trentaine de secondes. Ce "gyroscope virtuel" n'a donc pas été conservé dans la version finale : nous avons finalement réussi à nous procurer un véritable capteur physique. Nous avons même pu faire évoluer notre choix initial (le MPU-6050) vers un modèle **MPU-92.65** pour assurer un asservissement direct, stable et beaucoup plus fiable.

5.7 Obtention des données avec notre montage

Ce script a été spécifiquement développé pour caractériser la réponse dynamique du système. Il applique automatiquement un échelon de commande de **0.6V** et réalise une acquisition des données avec une période d'échantillonnage stricte de **50ms**. Ces paramètres permettent de tracer la courbe d'évolution temporelle avec une excellente résolution, facilitant ainsi l'analyse du régime transitoire et l'extraction des données de performance.

6. MatLab

6.1 Diagnostic et Identification du Système (Boucle Ouverte)

Avant de chercher à contrôler l'hélicoptère, il fallait comprendre sa dynamique naturelle. Nous avons injecté un échelon simple (1V) dans le modèle *State-Space* de la maquette, sans aucun correcteur.

L'analyse de cette réponse nous a permis d'identifier la Fonction de Transfert en Boucle Ouverte (FTBO) de notre système sous sa forme canonique du second ordre :

Hp = (0,08)/(2,48p^2 + 0,47p +1)

Cette équation traduit parfaitement la "personnalité" physique de la machine :

- **Un système "lourd" :** Le gain statique très faible ($K=0,08$) indique qu'il faut fournir beaucoup d'énergie pour obtenir un mouvement significatif.
- **Un comportement très oscillant :** Le système présente un sous-amortissement chronique ($\varepsilon \approx 0,15$). Naturellement, le bras manque de friction et oscille longuement avant de se stabiliser.
- **Une dynamique lente :** La pulsation propre $\omega 0$ de 0,635 rad/s confirme une inertie importante.

$$H(p) = \frac{K}{\frac{1}{\omega_0^2} \cdot p^2 + \frac{2 \cdot \xi}{\omega_0} \cdot p + 1}$$

6.2 Synthèse du Correcteur (La Compensation de Pôles)

Pour corriger ces défauts, nous avons utilisé la méthode mathématique de la **compensation de pôles**. L'objectif est de configurer le régulateur PID pour que son équation annule exactement les termes gênants du dénominateur de la machine (ceux qui causent les oscillations, soit les coefficients 0,47 et 1).

Ensuite, nous avons appliqué un gain global (A=18,75) pour forcer le système à atteindre sa cible en environ 2 secondes. Cette chirurgie mathématique nous a donné des gains réalistes et optimisés, compatibles avec nos actionneurs réels :

- **Gain Proportionnel (Kp) :** 8,8
- **Gain Intégral (Ki) :** 18,75
- **Gain Dérivé (Kd) :** 46,5

6.3 Validation et Analyse des Résultats

Nous avons ensuite simulé l'impact de ce nouveau régulateur PID sur deux modèles distincts pour valider notre approche.

1. Sur la Fonction de Transfert Théorique :

Lorsqu'on applique le PID à notre fonction de transfert mathématique $H(p)$, la réponse est parfaite. La courbe monte rapidement, atteint la consigne exacte de 1,0 sans aucun dépassement, et se stabilise instantanément. Dans ce monde mathématique idéal, le PID a parfaitement annulé les pôles oscillants du système.

2. Sur le Modèle "State-Space" Réel :

Nous avons ensuite appliqué ce même PID au modèle State-Space complet (qui inclut des comportements plus proches de la réalité de notre maquette).

Contrairement à nos premières simulations avec des gains extrêmes (qui plafonnaient à 0,87), **le système atteint cette fois-ci parfaitement la consigne de 1,0**. L'erreur statique a totalement disparu.

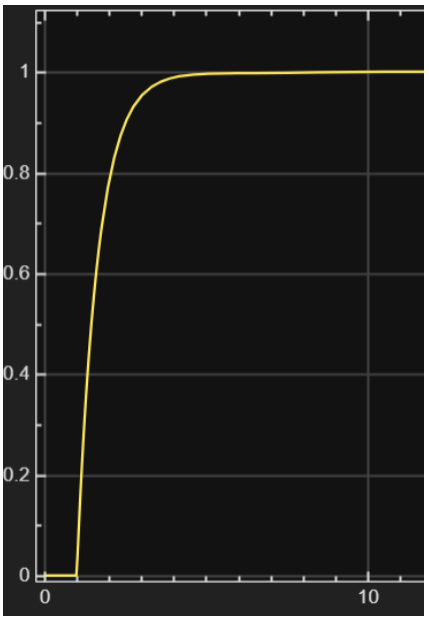
Nous observons cependant quelques oscillations résiduelles au démarrage. Celles-ci s'expliquent par trois facteurs inhérents au passage de la théorie à la pratique :

- **La complexité du modèle :** Le modèle State-Space contient des petites non-linéarités que notre fonction de transfert simplifiée du second ordre n'a pas pu capter à 100 %.
- **La Saturation (Windup) :** Lors de la montée brusque, les moteurs buttent contre leurs limites physiques (alimentation de 12V), ce qui perturbe l'action initiale du PID.
- **Le filtre de la dérivée :** Le filtrage indispensable du gain Kd modifie légèrement le calcul en temps réel, empêchant l'annulation mathématique d'être absolument parfaite.

Cette démarche nous a permis de passer d'un système qui oscillait dans le vide ou plafonnait sous la contrainte, à un système asservi capable de suivre précisément sa consigne en compensant sa propre inertie physique.

Figure 1 : Réponse du modèle théorique (PID sur Fonction de Transfert) —>

Figure 2 : Réponse du modèle réel (PID sur State-Space) —>



7. Conclusion

Le projet "Hélice-31" a pleinement atteint son objectif principal : concevoir un démonstrateur technique attractif et transportable, parfaitement adapté pour être exposé lors des Journées Portes Ouvertes (JPO). Nous avons réussi à transformer un concept théorique d'asservissement sur deux axes en une maquette physique fonctionnelle et interactive.

D'un point de vue technique, le passage d'un pilotage direct depuis Simulink à une commande numérique embarquée autonome sur Arduino a été un succès. La mise en place d'un échantillonnage rigoureux via les interruptions matériels (timers) a permis de garantir la stabilité de notre correcteur PID. De plus, l'interface homme-machine conçue permet un réglage didactique en direct des coefficients (Kp, Ki, Kd), illustrant concrètement l'impact de l'asservissement à tout utilisateur.

Ce projet nous a également confrontés à la réalité de l'ingénierie matériel. Qu'il s'agisse de la gestion stricte de la sécurité au démarrage des moteurs Brushless , de la minimisation des frottements mécaniques , ou encore de l'adaptation face au manque de composants, chaque défi a été surmonté. L'intégration finale du capteur MPU-92.65 a couronné ces efforts en offrant un asservissement direct et fiable.

En perspective d'évolution, comme évoqué dans le cahier des charges, il serait intéressant de développer la bascule entre un correcteur analogique et numérique , ainsi que de finaliser l'intégration complète du pilotage par boîtier de commande radiofréquence.

8. Note de l'Équipe

Note personnelle de l'équipe projet

La réalisation de la maquette Hélice-31 a été une aventure technique et humaine extrêmement enrichissante pour l'ensemble de notre groupe (Tristan, Fatihhan, Nils, Loan, Louis, Victor et Arthur). Ce projet nous a permis de faire le pont entre les équations mathématiques pures vues en classe et le comportement physique réel des composants électroniques et mécaniques.

Nous tenons tout particulièrement à remercier nos professeurs, M. Trenchant et M. Schlinquer, pour leur accompagnement, leur supervision et de leur aide tout au long de ces 5 séances. Voir notre hélicoptère fonctionner après des heures de codage, de modélisation et de câblage restera l'une des plus belles récompenses de notre formation en Sciences de l'Ingénieur.